Lab 1 to 10

NAME: Pratip raj P

REG:192521409

EXP 1:

Aim

To write an R program to check whether a given number is positive, negative, or zero.

Procedure

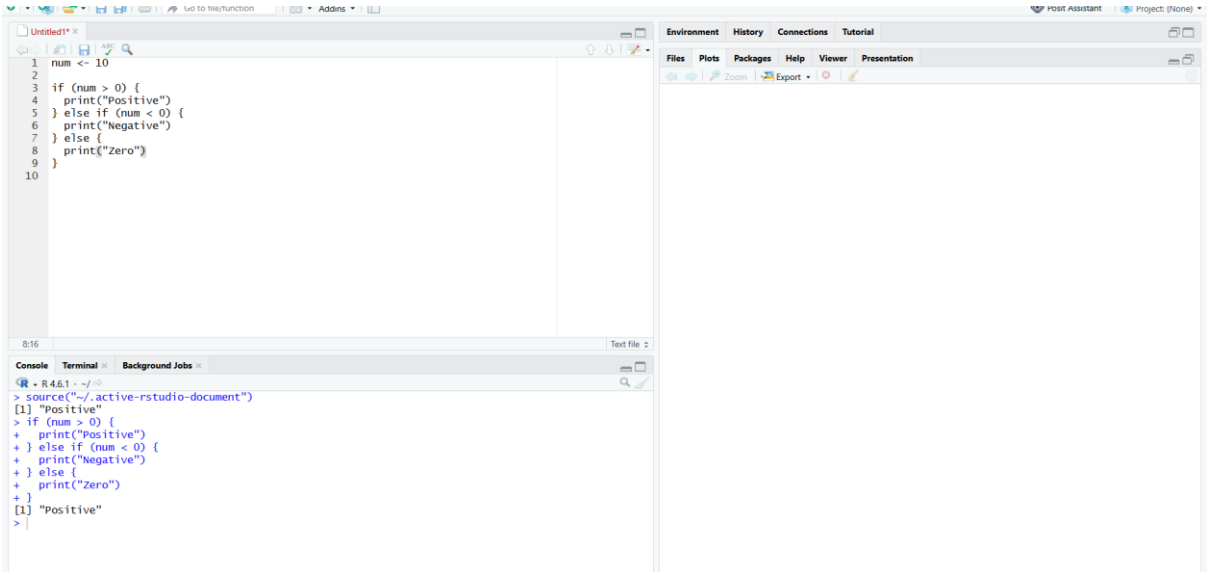
1. Start the RStudio program.
2. Assign a number to a variable.
3. Use if, else if, and else statements.
4. Check whether the number is positive, negative, or zero.
5. Display the result.

CODE:

```
num <- 10

if (num > 0) {
  print("Positive")
} else if (num < 0) {
  print("Negative")
} else {
  print("Zero")
}
```

RESULT:



EXP:2

Aim

To write an R program to generate the multiplication table of a given number.

Procedure

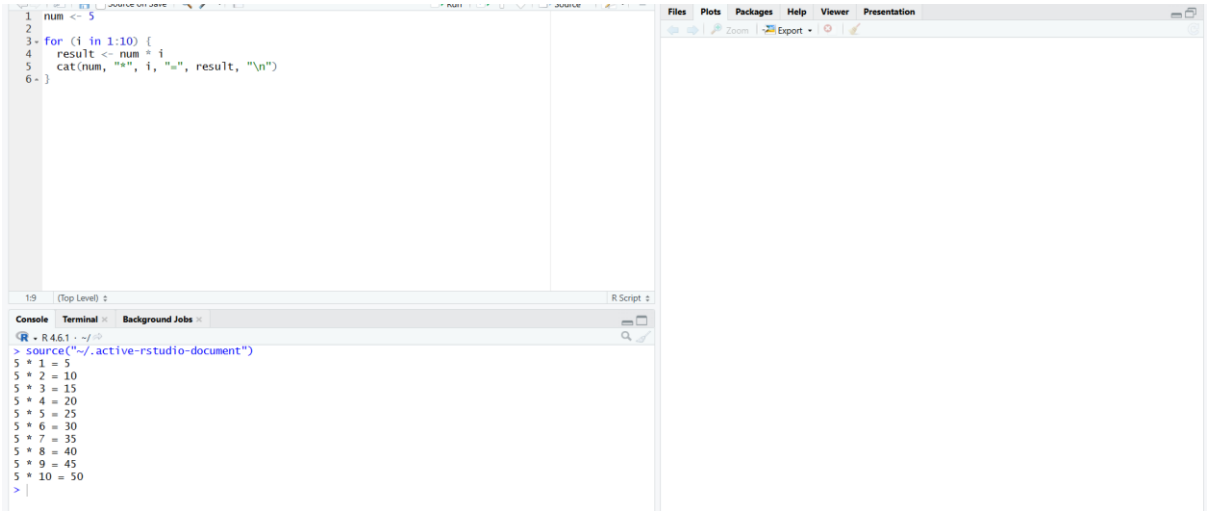
1. Assign a number to a variable.
2. Use a for loop from 1 to 10.
3. Multiply the given number by each value.
4. Display the multiplication table.

CODE:

```
num <- 5


for (i in 1:10) {
  result <- num * i
  cat(num, "*", i, "=", result, "\n")
}
```

RESULT:



EXP 3:

Aim

To write an R program to find the sum of digits of a given number.

Procedure

1. Assign a number to a variable.
2. Extract the last digit using %%.
3. Add the digit to the sum.
4. Remove the last digit using %/%.
5. Repeat until the number becomes zero.
6. Display the sum.

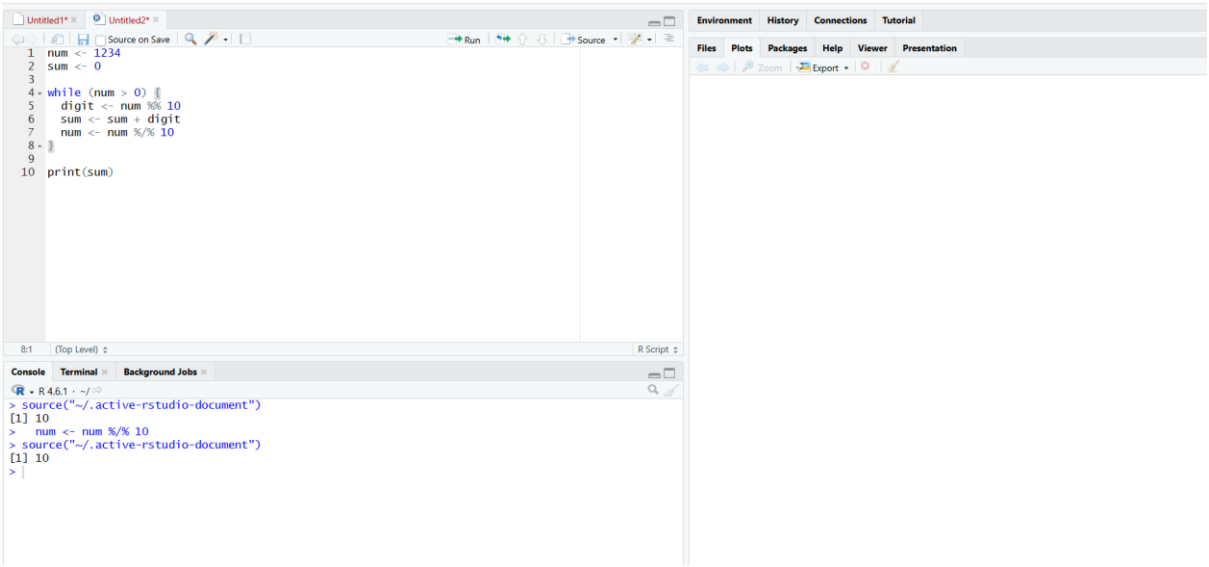
CODE:

```
num <- 1234
sum <- 0

while (num > 0) {
  digit <- num %% 10
  sum <- sum + digit
  num <- num %/% 10
}
```

print(sum)

RESULT:



EXP 4:

Aim

To write an R program to find the factorial of a number using recursion.

Procedure

1. Define a function named factorial_fun.
2. Check the base condition.
3. If the number is 1, return 1.
4. Otherwise multiply the number by the factorial of the previous number.
5. Call the function and display the result.

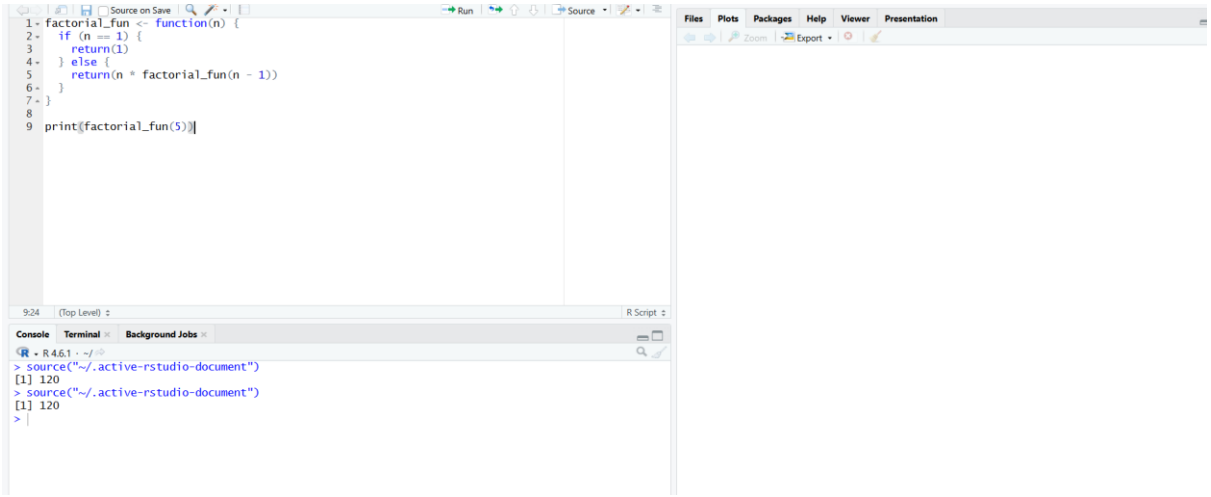
CODE:

```
factorial_fun <- function(n) {
  if (n == 1) {
    return(1)
  } else {
    return(n * factorial_fun(n - 1))
```

```
  }
}

print(factorial_fun(5))
```

RESULT:



EXP:5

Aim

To write an R program to generate the Fibonacci series using recursion.

Procedure

1. Define a recursive Fibonacci function.
2. Set the base conditions for 0 and 1.
3. Add the previous two Fibonacci values.
4. Use a for loop to display the series.
5. Display the result.

CODE:

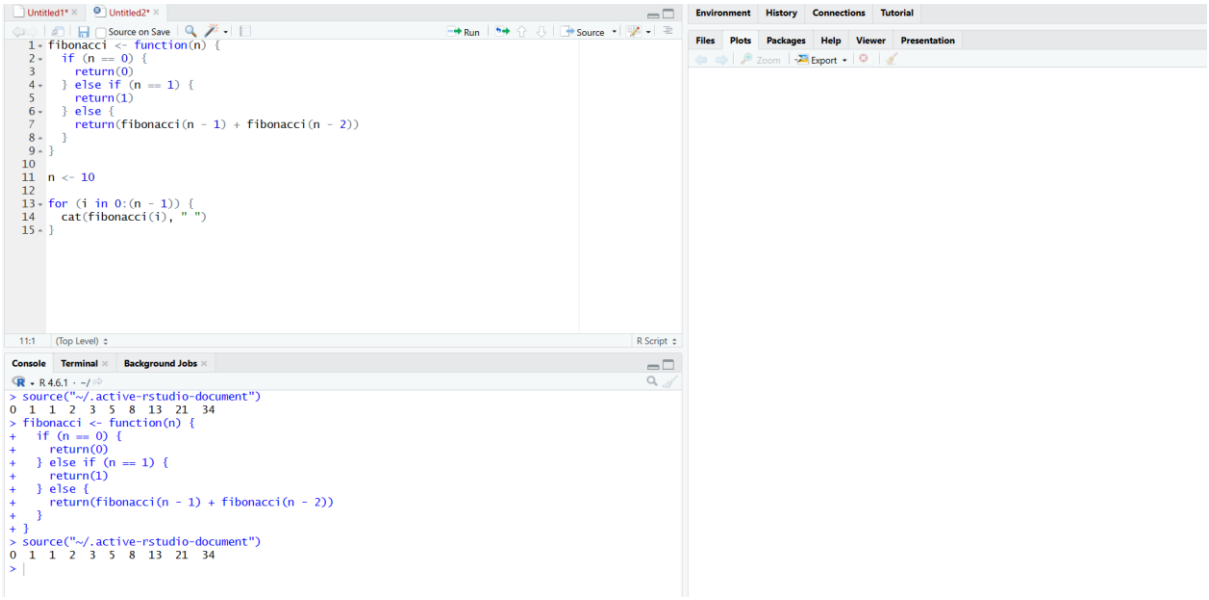
```
fibonacci <- function(n) {
  if (n == 0) {
    return(0)
  } else if (n == 1) {
```

```
    return(1)
  } else {
    return(fibonacci(n - 1) + fibonacci(n - 2))
  }
}

n <- 10

for (i in 0:(n - 1)) {
  cat(fibonacci(i), " ")
}
```

RESULT:



EXP 6:

Aim

To write an R program to find the factors of a given number.

Procedure

1. Assign a number to a variable.

2. Run a loop from 1 to the given number.
3. Check whether the number is exactly divisible by the loop value.
4. Store the divisible values as factors.
5. Display the factors.

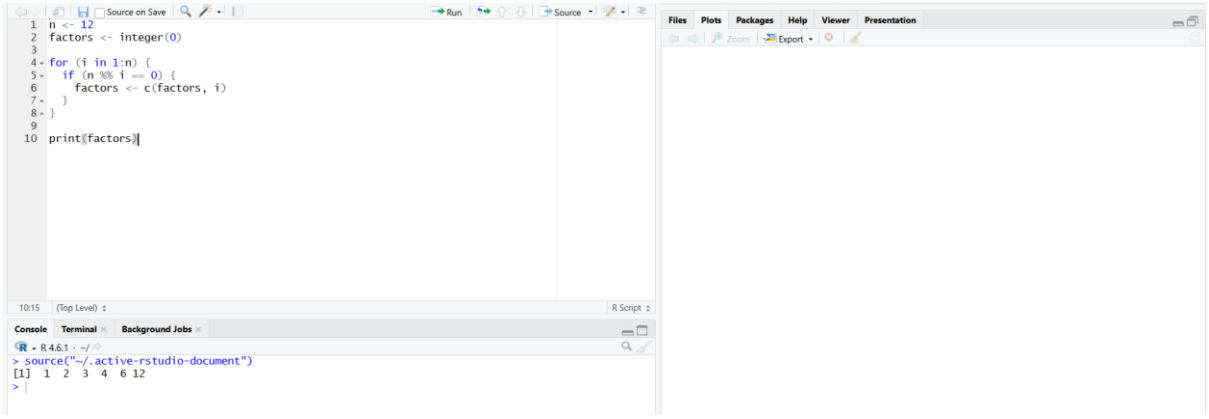
CODE:

```
n <- 12
factors <- integer(0)

for (i in 1:n) {
  if (n %% i == 0) {
    factors <- c(factors, i)
  }
}

print(factors)
```

RESULT:



EXP 7:

Aim

To write an R program to print FizzBuzz values from 1 to 100.

Procedure

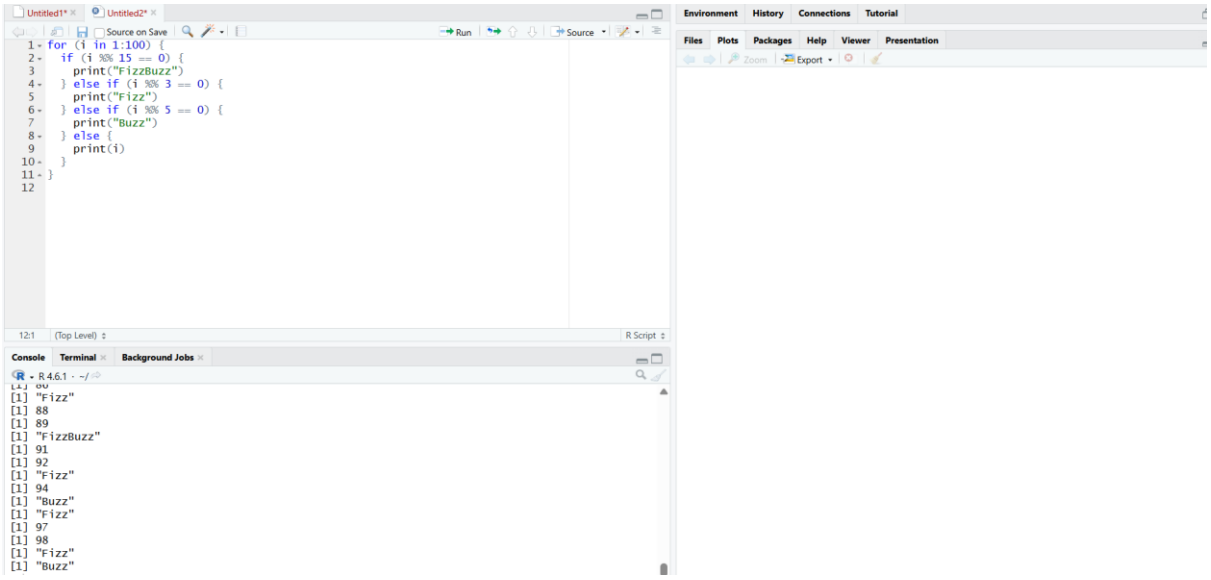
1. Use a for loop from 1 to 100.
2. Check numbers divisible by both 3 and 5.

3. Print FizzBuzz for those numbers.
4. Print Fizz for numbers divisible by 3.
5. Print Buzz for numbers divisible by 5.
6. Otherwise print the number.

CODE:

```
for (i in 1:100) {
  if (i %% 15 == 0) {
    print("FizzBuzz")
  } else if (i %% 3 == 0) {
    print("Fizz")
  } else if (i %% 5 == 0) {
    print("Buzz")
  } else {
    print(i)
  }
}
```

RESULT:



EXP 8:

Aim

To create numeric, character, and logical vectors and identify their classes in R.

Procedure

1. Create a numeric vector.
2. Create a character vector.
3. Create a logical vector.
4. Use the class() function.
5. Display the vectors and their classes.

CODE:

```
a <- c(1, 2, 3, 4, 5)

b <- c("Ram", "Sam", "Raj")

c <- c(TRUE, FALSE, TRUE)


print(a)

print(class(a))

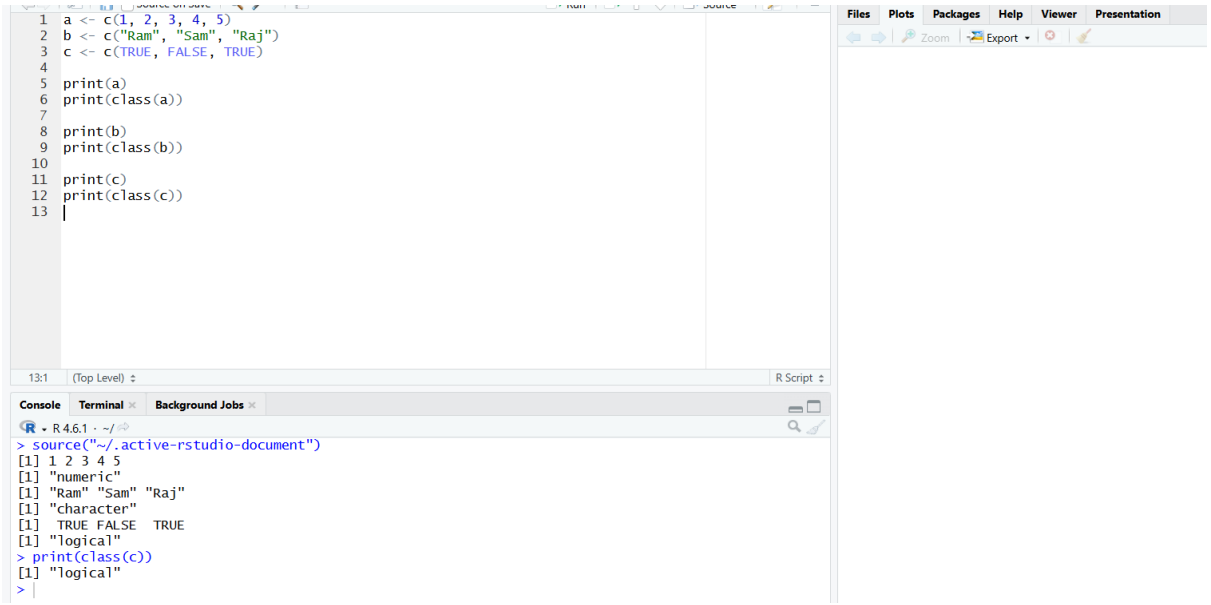

print(b)

print(class(b))
```

print(c)

print(class(c))

RESULT:



EXP 9:

Aim

To create and display a matrix using R.

Procedure

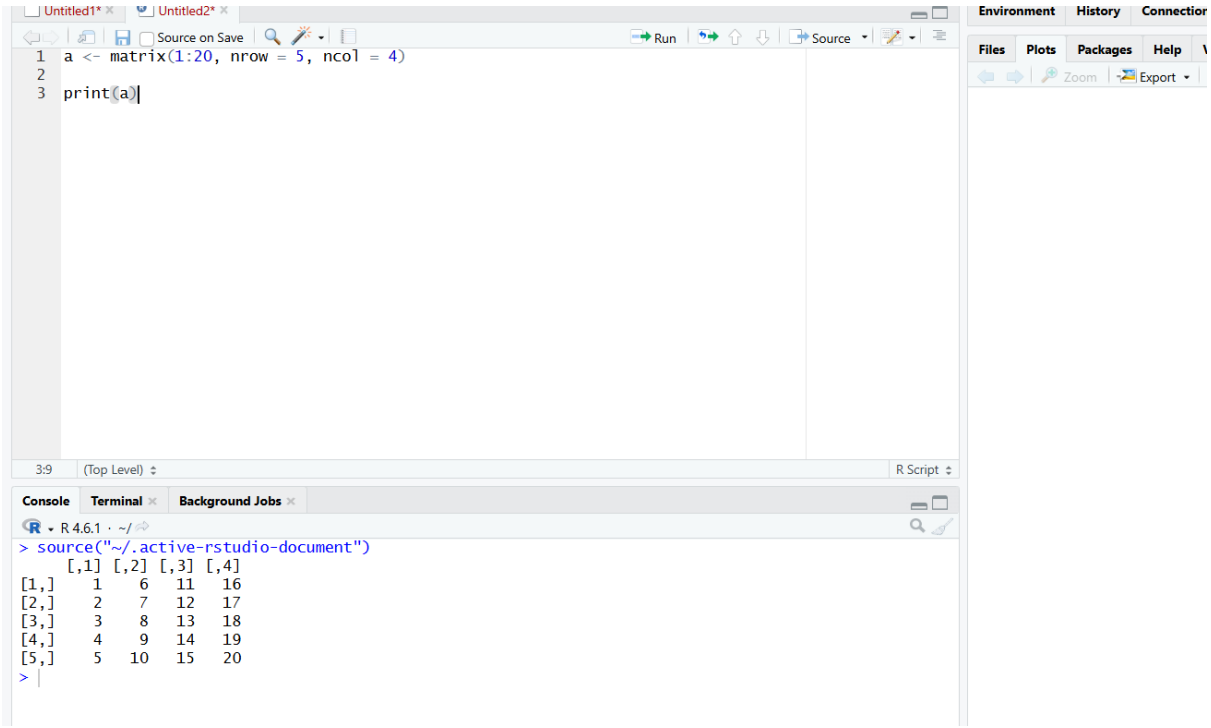
1. Use the matrix() function.
2. Generate numbers from 1 to 20.
3. Set the number of rows as 5.
4. Set the number of columns as 4.
5. Display the matrix.

CODE:

a <- matrix(1:20, nrow = 5, ncol = 4)

print(a)

RESULT:



EXP 10:

Aim

To create basic statistical graphs such as boxplot, histogram, bar plot, line plot, and scatter plot using R.

Procedure

1. Create x and y values.
2. Use boxplot() to create a boxplot.
3. Use hist() to create a histogram.
4. Use barplot() to create a bar graph.
5. Use plot() to create line and scatter plots.
6. Display the graphs.

CODE:

```
x <- c(10, 20, 30, 40, 50)

y <- c(5, 15, 25, 35, 45)


boxplot(x)
```

hist(x)

barplot(x)

plot(x, type = "l")

plot(x, y)

RESULT:

